

Aufgabenaufteilung fuer 3 Entwickler

Ziel: faire Verteilung mit klaren Zuständigkeiten und moeglichst wenig Ueberschneidung.

Kurzueberblick

Jeder Entwickler bekommt einen eigenen Bereich. Die gemeinsame Schnittstelle sind nur die Props zwischen Formular, Uebersicht und Zusatzkomponenten. So koennen alle parallel arbeiten, ohne sich staendig gegenseitig zu blockieren.

Entwickler 1 - Aufgabenuebersicht

Fokus: Alles rund um die Seite /tasks und die Anzeige vorhandener Aufgaben.

Aufgaben:

- Route /tasks erstellen.
- TaskOverviewView erstellen.
- GET http://localhost:8080/task beim Laden ausfuehren.
- Aufgaben im State speichern.
- LoadingState, ErrorMessage und EmptyState anzeigen.
- TaskList erstellen.
- Basis-TaskCard erstellen.
- Titel, Beschreibung, Faelligkeitsdatum und Status anzeigen.

User Story: Aufgabenuebersicht anzeigen

Entwickler 2 - Neue Aufgabe erstellen

Fokus: Alles rund um die Seite /tasks/new und das Grundformular zum Erstellen einer Aufgabe.

Aufgaben:

- Route /tasks/new erstellen.
- TaskForm erstellen.
- Felder fuer Titel, Beschreibung und Faelligkeitsdatum anlegen.
- Formular-State verwalten.
- Pflichtfelder validieren: Titel und Faelligkeitsdatum.
- Beim Absenden status = OFFEN setzen.
- POST http://localhost:8080/task senden.
- Speicher-Loading und Speicher-Fehler anzeigen.
- Nach erfolgreichem Speichern zu /tasks navigieren.

User Story: #15 Neue Aufgabe mit Deadline erstellen

Entwickler 3 - Zuweisung, Rotation und Statuslogik

Fokus: Erweiterungen, die auf Uebersicht und Formular aufbauen, aber als eigene Komponenten und Hilfsfunktionen umgesetzt werden.

Aufgaben:

- GET http://localhost:8080/person anbinden.
- AssigneeSelect fuer genau einen Mitbewohner erstellen.
- personId im Formular-State ergaenzen und beim POST mitsenden.
- RotationSettings erstellen.
- MultiPersonSelect fuer mehrere Mitbewohner erstellen.

- rotationAktiv, wiederholungsIntervall und rotationsPersonIds verwalten.
- Validierung: bei aktiver Rotation mindestens zwei Personen auswaehlen.
- isOverdue(task) erstellen.
- Ueberfaellig-Badge und auffaellige Darstellung ergaenzen.
- Erledigen-Aktion per PATCH /task/{id}/status umsetzen.

User Story: #18 Zuweisung, #19 Rotation, Aufgabe erledigen, #25 Ueberfaellige Aufgaben

Schnittstellen zwischen den Entwicklern

Bereich	Absprache
Task-Objekt	Gemeinsame Felder: id, titel, beschreibung, faelligkeitsdatum, status, personId, rotationAktiv, wiederholungsIntervall, rotationsPersonIds.
TaskForm	Entwickler 2 baut das Grundformular. Entwickler 3 ergaenzt AssigneeSelect und RotationSettings ueber klar getrennte Komponenten.
TaskCard	Entwickler 1 baut die Basisanzeige. Entwickler 3 ergaenzt Erledigen-Button und Ueberfaellig-Anzeige.
API	GET /task fuer Uebersicht, POST /task fuer Erstellung, GET /person fuer Mitbewohner, PATCH /task/{id}/status fuer Erledigt.

Empfohlene Reihenfolge

- Entwickler 1 startet mit /tasks, TaskOverviewView, TaskList und Basis-TaskCard.
- Entwickler 2 startet parallel mit /tasks/new, TaskForm, Validierung und POST /task.
- Entwickler 3 startet mit GET /person und AssigneeSelect, weil das frueh im Formular gebraucht wird.
- Danach ergaenzt Entwickler 3 RotationSettings und MultiPersonSelect.
- Zum Schluss integriert Entwickler 3 Erledigen-Aktion und Ueberfaellig-Logik in die Uebersicht.

Wichtig fuer wenig Ueberschneidung

- Nicht gleichzeitig dieselbe Datei bearbeiten, wenn es vermeidbar ist.
- TaskForm und TaskCard frueh mit Props vorbereiten, damit Erweiterungen sauber eingehaengt werden koennen.
- Gemeinsam kurz festlegen, ob die Zuweisung personId Pflicht ist.
- Backend-Endpunkte und Feldnamen vor der Integration einmal gemeinsam pruefen.

Definition of Done fuer das Team

- Normale Aufgaben koennen angezeigt und erstellt werden.
- Loading, Error und EmptyState sind sichtbar umgesetzt.
- Aufgaben koennen einem Mitbewohner zugewiesen werden.
- Wiederkehrende Aufgaben koennen mit Rotation erstellt werden.
- Offene Aufgaben koennen als erledigt markiert werden.
- Ueberfaellige Aufgaben werden klar markiert, erledigte Aufgaben nie.
- Keine produktive Anzeige verwendet Dummy-Daten.